

CHƯƠNG 07: TÁC NHÂN LOGIC

TRÍ TUỆ NHÂN TẠO - ARTIFICIAL INTELLIGENCE

Nội dung Chương 7

- ◇ 7.1 Tác nhân dựa trên Tri thức (Knowledge-Based Agents)
- ◇ 7.2 Thế giới Wumpus (The Wumpus World)
- ◇ 7.3 Logic học cơ bản (Logic)
- ◇ 7.4 Logic Mệnh đề (Propositional Logic)
- ◇ 7.5 Chứng minh định lý (Propositional Theorem Proving)
- ◇ 7.6 Tác nhân Logic Mệnh đề (Agents Based on Propositional Logic)

7.1 Tác nhân dựa trên Tri thức

◇ Khác với các tác nhân tìm kiếm chỉ biết mô phỏng các bước đi, **Tác nhân dựa trên Tri thức (Knowledge-Based Agents - KBA)** có khả năng "suy nghĩ" và "lập luận" dựa trên những gì chúng biết.

◇ **Thành phần cốt lõi:**

- **Cơ sở tri thức (Knowledge Base - KB):** Là một tập hợp các Câu (Sentences) biểu diễn những sự thật về thế giới. Mỗi câu được viết bằng một **Ngôn ngữ biểu diễn tri thức** (Knowledge Representation Language).
- **Cơ chế suy diễn (Inference Engine):** Hệ thống giúp suy ra các sự thật logic mới từ những thông tin đã có trong KB.

7.1 Tác nhân dựa trên Tri thức (Tiếp theo)

◇ Hai thao tác chính của KB:

- TELL: Bơm thêm tri thức mới (hoặc thông tin nhận biết từ môi trường) vào cơ sở tri thức.
- ASK: Đặt câu hỏi truy vấn để tác nhân quyết định xem nên thực hiện hành động nào tiếp theo.

Kiến trúc Tác nhân Tri thức

function KB-AGENT(*percept*) **returns** an *action*
persistent: *KB*, a knowledge base
t, a counter, initially 0, indicating time

TELL(*KB*, MAKE-PERCEPT-SENTENCE(*percept*, *t*))
action $\leftarrow$ ASK(*KB*, MAKE-ACTION-QUERY(*t*))
TELL(*KB*, MAKE-ACTION-SENTENCE(*action*, *t*))
t $\leftarrow$ *t* + 1
return *action*

Figure 7.1: Tác nhân KBA nhận thông tin từ Môi trường, Suy diễn và Ra lệnh hành động.

7.2 Thế giới Wumpus (Wumpus World)

◇ Wumpus World là một môi trường giả lập kinh điển để kiểm tra khả năng lập luận logic của AI. Đây là một hang động có dạng lưới 4×4 .

◇ **Luật chơi:**

- **Quái vật Wumpus:** Nằm ẩn nấp trong 1 ô. Nếu tác nhân bước vào, sẽ bị an thịt (Game Over). Wumpus phát ra mùi **Stench (Mùi hôi)** ở các ô kề cạnh.
- **Hố sâu (Pits):** Rải rác khắp hang. Bước vào là chết. Hố sâu tạo ra luồng **Breeze (Gió nhẹ)** ở các ô kề cạnh.
- **Vàng (Gold):** Mục tiêu của tác nhân là tìm thỏi vàng. Thỏi vàng phát ra tia **Glitter (Sáng lấp lánh)** ở chính ô của nó.
- Tác nhân có 1 mũi tên duy nhất để bắn chết Wumpus. Nếu Wumpus chết sẽ phát ra tiếng **Scream (Tiếng hét)**.

Mô phỏng Thế giới Wumpus

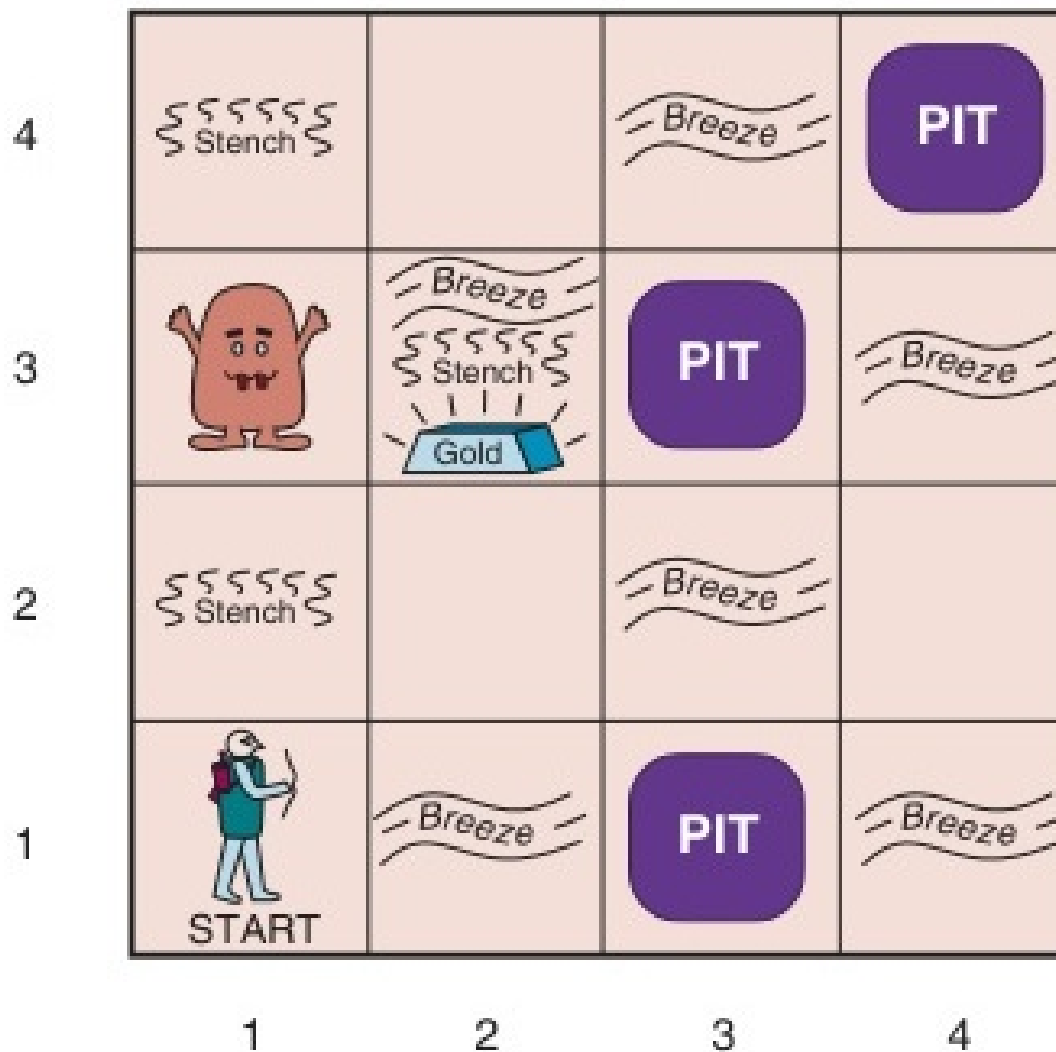


Figure 7.2: Môi trường Wumpus World với Wumpus, Vàng và các Hồ sâu.

Khám phá Thế giới Wumpus

◇ Tại sao Tìm kiếm (Search) thông thường thất bại trong Wumpus? Vì môi trường này là **Quan sát một phần (Partially observable)**.

Tác nhân không biết trước bản đồ.

◇ **Cách Tác nhân Logic hoạt động:**

- Bước vào ô [1, 1]: Tác nhân không ngửi thấy gì, không cảm thấy gió.
- Suy ra (Inference): Các ô kề cạnh [1, 2] và [2, 1] tuyệt đối an toàn.
- Bước sang ô [2, 1]: Tác nhân cảm thấy **Gió nhẹ (Breeze)**.
- Suy luận logic: "Chắc chắn có ít nhất 1 Hố sâu ở [2, 2] hoặc [3, 1]".
- Nhờ Logic, tác nhân có thể **chứng minh được sự an toàn** của một ô trước khi mạo hiểm bước vào!

Bước đầu tiên trong Thế giới Wumpus

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK			
1,1	2,1	3,1	4,1
A			
OK	OK		

(a)

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK	P?		
1,1	2,1	3,1	4,1
V	A	P?	
OK	B		
	OK		

(b)

Figure 7.3: Bước đầu tiên được thực hiện bởi tác tử trong thế giới wumpus.

Quá trình Kết hợp Cảm nhận

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

(a)

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4 P?	3,4	4,4
1,3 W!	2,3 A S G B	3,3 P?	4,3
1,2 S V OK	2,2 V OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

(b)

Figure 7.4: Tác nhân kết hợp các cảm nhận (Breeze, Stench) để chốt vị trí Wumpus.

7.3 Logic học cơ bản

◇ **Logic** bao gồm 2 yếu tố: Cú pháp (Syntax) và Ngữ nghĩa (Semantics).

◇ **Kéo theo logic (Entailment)**: Kí hiệu là $\models$.

$$\alpha \models \beta$$

Đọc là: *Câu α kéo theo câu β* . Nghĩa là trong mọi thế giới (mô hình) mà α đúng, thì β cũng bắt buộc phải đúng.

(VD: "A là một con mèo" $\models$ "A là một động vật").

◇ **Suy diễn thuật toán (Inference)**: Kí hiệu là $\vdash$.

$$KB \vdash_i \alpha$$

Nghĩa là câu α có thể được suy ra từ Cơ sở tri thức (KB) bằng một thuật toán i . Thuật toán lý tưởng phải đạt tính **Lành mạnh (Soundness)** và **Hoàn thiện (Completeness)**.

Mô hình Thế giới (Models)

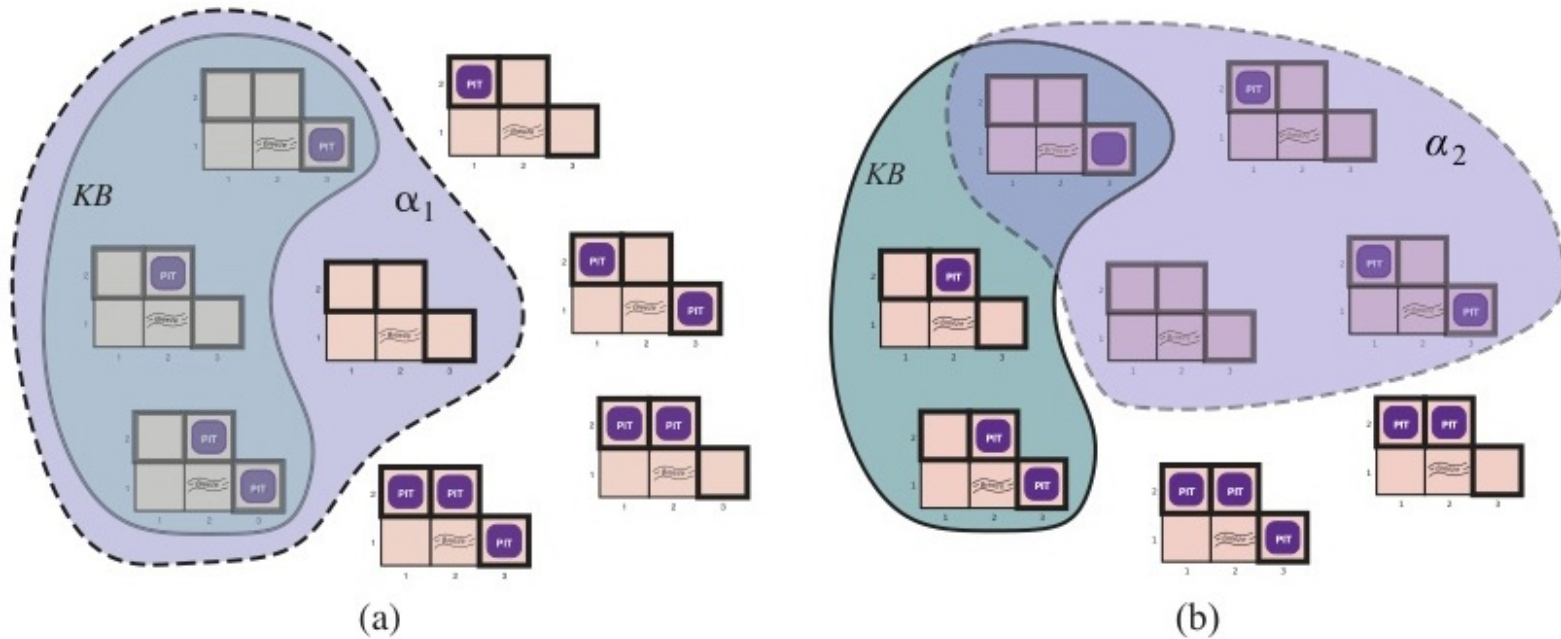


Figure 7.5: Entailment: Tập hợp các thế giới thỏa mãn KB hoàn toàn bao hàm tập hợp các thế giới thỏa mãn kết luận α .

Quá trình Suy luận Cơ bản

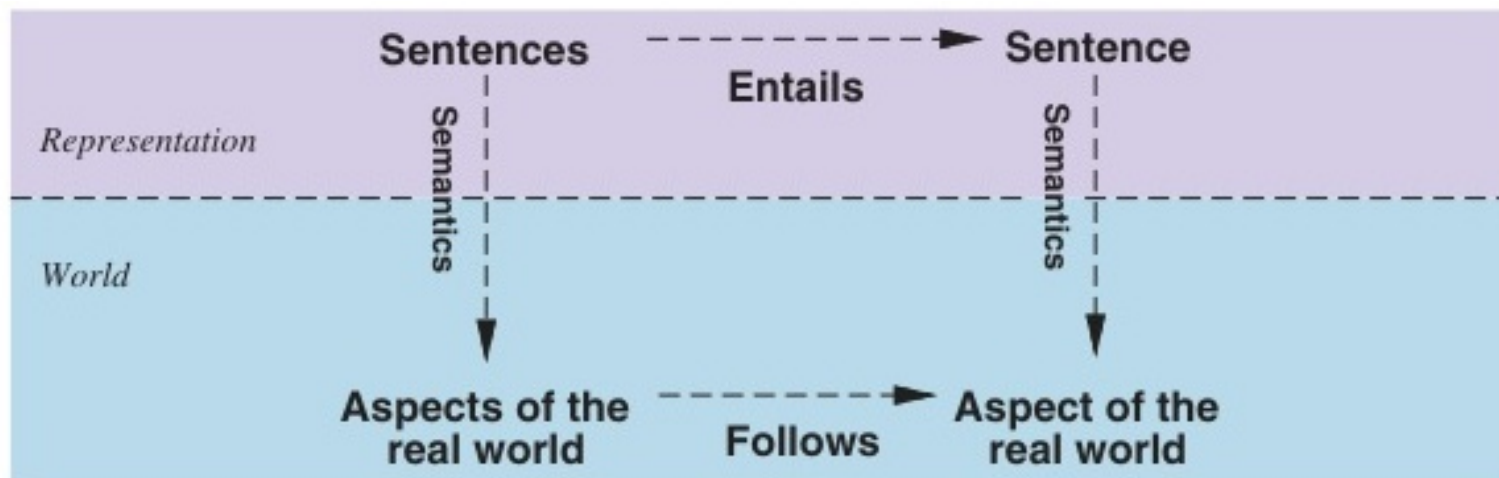


Figure 7.6: Các câu là các cấu hình vật lý của tác tử, và suy luận xây dựng cấu hình mới từ cũ.

7.4 Logic Mệnh đề (Propositional Logic)

◇ Logic Mệnh đề là ngôn ngữ logic đơn giản nhất. Mỗi mệnh đề là một sự thật mang giá trị Đúng (True - T) hoặc Sai (False - F).

◇ **Các phép toán logic cơ bản:**

- Phủ định (NOT): $\neg P$
- Hội (AND): $P \wedge Q$ (Chỉ đúng khi cả P và Q đều đúng)
- Tuyển (OR): $P \vee Q$ (Đúng khi P đúng, hoặc Q đúng, hoặc cả hai)
- Kéo theo (IMPLIES): $P \Rightarrow Q$ (Chỉ sai khi P đúng nhưng Q sai)
- Tương đương (IFF): $P \Leftrightarrow Q$ (Đúng khi P và Q có cùng giá trị)

◇ Từ các kí hiệu này, ta có thể biểu diễn toàn bộ luật lệ của thế giới Wumpus vào Cơ sở tri thức! (Ví dụ: "Không có hồ ở ô [1, 1]" viết là $\neg P_{1,1}$).

Ngữ pháp Logic Mệnh đề

$$\begin{aligned} \textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\ \textit{AtomicSentence} &\rightarrow \textit{True} \mid \textit{False} \mid P \mid Q \mid R \mid \dots \\ \textit{ComplexSentence} &\rightarrow (\textit{Sentence}) \\ &\mid \neg \textit{Sentence} \\ &\mid \textit{Sentence} \wedge \textit{Sentence} \\ &\mid \textit{Sentence} \vee \textit{Sentence} \\ &\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\ &\mid \textit{Sentence} \Leftrightarrow \textit{Sentence} \end{aligned}$$

OPERATOR PRECEDENCE : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Figure 7.7: Ngữ pháp BNF (Dạng Backus–Naur) của các câu trong logic mệnh đề.

Các Tương đương Logic Chuẩn

$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$	commutativity of $\wedge$
$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$	commutativity of $\vee$
$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$	associativity of $\wedge$
$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$	associativity of $\vee$
$\neg(\neg\alpha) \equiv \alpha$	double-negation elimination
$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$	contraposition
$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$	implication elimination
$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$	biconditional elimination
$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$	De Morgan
$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$	De Morgan
$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$	distributivity of $\wedge$ over $\vee$
$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$	distributivity of $\vee$ over $\wedge$

Figure 7.8: Danh sách các quy tắc biến đổi tương đương logic (VD: De Morgan).

Bảng Chân trị (Truth Table)

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>
<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>
<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>
<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>

Figure 7.9: Bảng chân trị liệt kê mọi trường hợp Đúng/Sai của các mệnh đề phức hợp.

Bảng Chân trị cho Cơ sở Tri thức

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>false</i>
<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>
<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<u><i>true</i></u>
<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<u><i>true</i></u>
<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<u><i>true</i></u>
<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>false</i>

Figure 7.10: Một bảng chân lý được xây dựng để kiểm tra tính thỏa mãn của KB thế giới Wumpus.

Thuật toán Kiểm tra Bảng chân trị

function TT-ENTAILS?(KB, α) **returns** *true* or *false*

inputs: KB , the knowledge base, a sentence in propositional logic
 α , the query, a sentence in propositional logic

$symbols \leftarrow$ a list of the proposition symbols in KB and α

return TT-CHECK-ALL($KB, \alpha, symbols, \{ \}$)

function TT-CHECK-ALL($KB, \alpha, symbols, model$) **returns** *true* or *false*

if EMPTY?($symbols$) **then**

if PL-TRUE?($KB, model$) **then return** PL-TRUE?($\alpha, model$)

else return *true* // when KB is false, always return *true*

else

$P \leftarrow$ FIRST($symbols$)

$rest \leftarrow$ REST($symbols$)

return (TT-CHECK-ALL($KB, \alpha, rest, model \cup \{P = true\}$)

and

 TT-CHECK-ALL($KB, \alpha, rest, model \cup \{P = false\}$))

Figure 7.11: Một thuật toán liệt kê bảng chân lý để quyết định sự bao hàm mệnh đề (TT-ENTAILS).

7.5 Chứng minh Định lý Mệnh đề

◇ Làm sao để máy tính tự động chứng minh một kết luận α là đúng dựa trên KB?

◇ **Phương pháp 1: Kiểm tra mô hình (Model Checking)**

- Lập bảng chân trị liệt kê **tất cả** 2^n thế giới có thể xảy ra.
- Kiểm tra xem có đúng là trong mọi dòng mà KB đúng, thì α cũng đúng hay không.
- Nhược điểm: Cực kì chậm, độ phức tạp hàm mũ $\mathcal{O}(2^n)$.

◇ **Phương pháp 2: Suy diễn mệnh đề (Theorem Proving)**

- Áp dụng các quy tắc biến đổi đại số logic (Modus Ponens, Thuật toán Phân giải) trực tiếp trên các phương trình.

Thuật toán Phân giải (Resolution)

◇ Trước khi áp dụng Phân giải, mọi câu mệnh đề phải được chuyển về **Dạng chuẩn Tắc (Conjunctive Normal Form - CNF)**.

◇ CNF là dạng Hội của các Tuyến: $(A \vee B) \wedge (\neg B \vee C \vee D) \wedge \dots$

◇ **Quy tắc phân giải cơ bản:** Nếu ta có 2 mệnh đề: $(P \vee Q)$ và $(\neg Q \vee R)$.

Ta có thể triệt tiêu Q và $\neg Q$ để suy ra trực tiếp câu mới: $(P \vee R)$.

◇ **Chứng minh bằng Phản chứng (Proof by Contradiction):**

Để chứng minh $KB \models \alpha$, ta giả sử ngược lại $\neg\alpha$ là đúng. Ta nhét $\neg\alpha$ vào KB và chạy thuật toán Phân giải. Nếu ta triệt tiêu được mọi thứ và sinh ra mệnh đề Rỗng (Điều vô lý), thì kết luận α ban đầu chắc chắn là Đúng!

Dạng Chuẩn Tắc CNF

$CNFSentence \rightarrow Clause_1 \wedge \dots \wedge Clause_n$

$Clause \rightarrow Literal_1 \vee \dots \vee Literal_m$

$Fact \rightarrow Symbol$

$Literal \rightarrow Symbol \mid \neg Symbol$

$Symbol \rightarrow P \mid Q \mid R \mid \dots$

$HornClauseForm \rightarrow DefiniteClauseForm \mid GoalClauseForm$

$DefiniteClauseForm \rightarrow Fact \mid (Symbol_1 \wedge \dots \wedge Symbol_l) \Rightarrow Symbol$

$GoalClauseForm \rightarrow (Symbol_1 \wedge \dots \wedge Symbol_l) \Rightarrow False$

Figure 7.12: Một ngữ pháp cho dạng chuẩn hội (CNF) và mệnh đề Horn.

Ví dụ Thuật toán Phân giải CNF

function PL-RESOLUTION(KB, α) **returns** *true* or *false*
inputs: KB , the knowledge base, a sentence in propositional logic
 α , the query, a sentence in propositional logic

$clauses \leftarrow$ the set of clauses in the CNF representation of $KB \wedge \neg\alpha$
 $new \leftarrow \{ \}$
while *true* **do**
 for each pair of clauses C_i, C_j **in** $clauses$ **do**
 $resolvents \leftarrow$ PL-RESOLVE(C_i, C_j)
 if $resolvents$ contains the empty clause **then return** *true*
 $new \leftarrow new \cup resolvents$
 if $new \subseteq clauses$ **then return** *false*
 $clauses \leftarrow clauses \cup new$

Figure 7.13: Cây phân giải: Triệt tiêu các cặp giá trị ngược dấu để sinh mệnh đề Rỗng.

Ứng dụng Thuật toán Phân giải

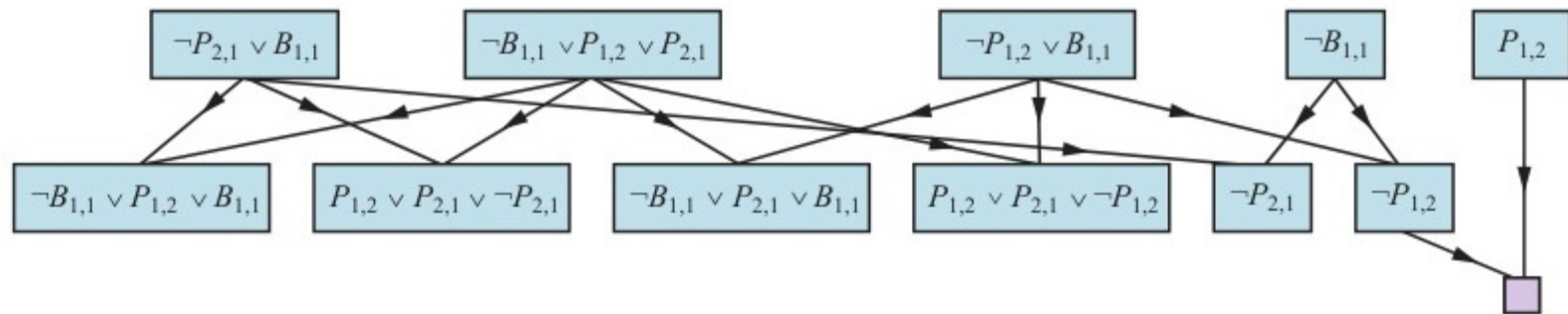


Figure 7.14: Ứng dụng PL-RESOLUTION cho một suy luận đơn giản trong thế giới wumpus.

Mệnh đề Horn và Suy diễn

◇ **Mệnh đề Horn:** Là câu chỉ chứa tối đa 1 literal dương. Rất dễ để suy diễn nhanh thông qua đồ thị logic AND-OR.

```
function PL-FC-ENTAILS?(KB, q) returns true or false
  inputs: KB, the knowledge base, a set of propositional definite clauses
           q, the query, a proposition symbol
  count ← a table, where count[c] is initially the number of symbols in clause c's premise
  inferred ← a table, where inferred[s] is initially false for all symbols
  queue ← a queue of symbols, initially symbols known to be true in KB

  while queue is not empty do
    p ← POP(queue)
    if p = q then return true
    if inferred[p] = false then
      inferred[p] ← true
      for each clause c in KB where p is in c.PREMISE do
        decrement count[c]
        if count[c] = 0 then add c.CONCLUSION to queue
  return false
```

Figure 7.15: Thuật toán suy diễn tiến cho logic mệnh đề.

7.6 - 7.7 Tác nhân Logic Mệnh đề

◇ Dựa trên Logic Mệnh đề, ta có thể xây dựng một Tác nhân Wumpus hoạt động hoàn chỉnh:

- **Cảm biến thu nhận:** Khi đến ô $[2, 1]$, cập nhật $B_{2,1} = T$ (Cảm nhận có Gió ở 2,1).
- **Luật lệ vật lý:** Khai báo quy tắc "Nếu ô có Gió, thì ít nhất 1 trong 4 ô kề cạnh phải có Hố" $\Rightarrow B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1} \vee P_{1,2})$.
- **Hành động suy luận:** Gọi cơ chế Phân giải để tự động suy luận ra xem $P_{2,2}$ an toàn hay không, và xuất ra quyết định đi tiếp.

◇ **Hạn chế của Logic mệnh đề:** Không thể khái quát hóa! Ta phải viết tay công thức Gió cho **toàn bộ** 16 ô trên bản đồ. Nếu bản đồ có kích thước 1000×1000 , số lượng phương trình mệnh đề sẽ dài hàng triệu dòng!

Đồ thị Mạch điện Logic (Logic Circuits)

```
function PL-FC-ENTAILS?(KB, q) returns true or false
  inputs: KB, the knowledge base, a set of propositional definite clauses
           q, the query, a proposition symbol
  count  $\leftarrow$  a table, where count[c] is initially the number of symbols in clause c's premise
  inferred  $\leftarrow$  a table, where inferred[s] is initially false for all symbols
  queue  $\leftarrow$  a queue of symbols, initially symbols known to be true in KB

  while queue is not empty do
    p  $\leftarrow$  POP(queue)
    if p = q then return true
    if inferred[p] = false then
      inferred[p]  $\leftarrow$  true
      for each clause c in KB where p is in c.PREMISE do
        decrement count[c]
        if count[c] = 0 then add c.CONCLUSION to queue
  return false
```

Figure 7.16: Thuật toán suy diễn tiến trong logic mệnh đề (Hoạt động giống Mạch điện tử logic AND/OR gates).

Bộ giải SAT và Thuật toán DPLL

◇ Bài toán SAT là NP-đầy đủ, nhưng thuật toán DPLL có thể giải quyết các bài toán khổng lồ nhờ kỹ thuật cắt tỉa thông minh.

function DPLL-SATISFIABLE?(*s*) **returns** *true* or *false*

inputs: *s*, a sentence in propositional logic

clauses $\leftarrow$ the set of clauses in the CNF representation of *s*

symbols $\leftarrow$ a list of the proposition symbols in *s*

return DPLL(*clauses*, *symbols*, { })

function DPLL(*clauses*, *symbols*, *model*) **returns** *true* or *false*

if every clause in *clauses* is true in *model* **then return** *true*

if some clause in *clauses* is false in *model* **then return** *false*

P, *value* $\leftarrow$ FIND-PURE-SYMBOL(*symbols*, *clauses*, *model*)

if *P* is non-null **then return** DPLL(*clauses*, *symbols* – *P*, *model* $\cup$ {*P*=*value*})

P, *value* $\leftarrow$ FIND-UNIT-CLAUSE(*clauses*, *model*)

if *P* is non-null **then return** DPLL(*clauses*, *symbols* – *P*, *model* $\cup$ {*P*=*value*})

P $\leftarrow$ FIRST(*symbols*); *rest* $\leftarrow$ REST(*symbols*)

return DPLL(*clauses*, *rest*, *model* $\cup$ {*P*=*true*}) **or**

DPLL(*clauses*, *rest*, *model* $\cup$ {*P*=*false*})

Figure 7.17: Thuật toán DPLL để kiểm tra tính thỏa mãn của một câu logic mệnh đề.

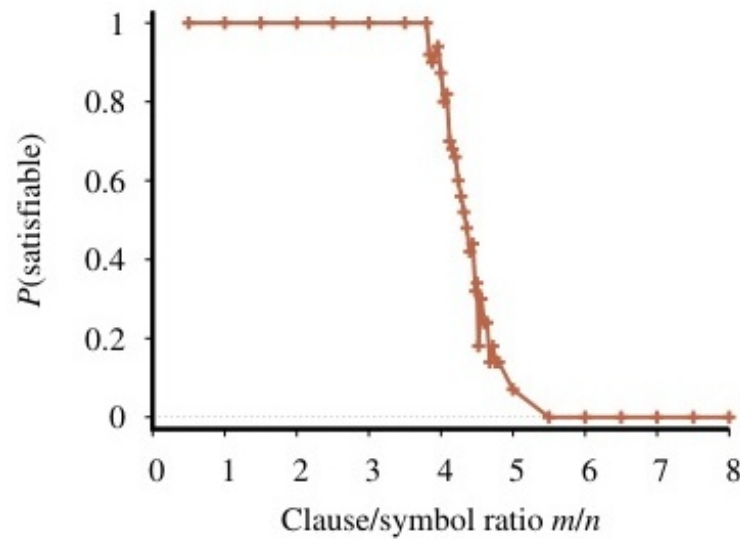
Tìm kiếm Cục bộ: WALKSAT

function WALKSAT(*clauses*, *p*, *max_flips*) **returns** a satisfying model or *failure*
inputs: *clauses*, a set of clauses in propositional logic
 p, the probability of choosing to do a “random walk” move, typically around 0.5
 max_flips, number of value flips allowed before giving up

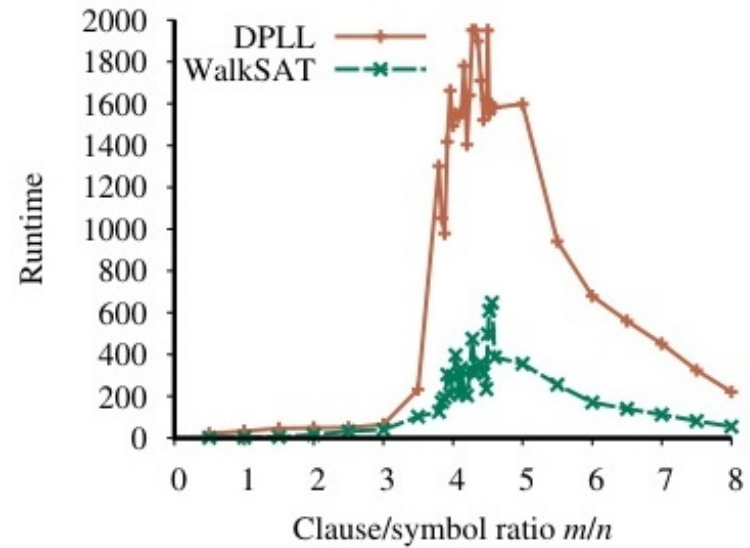
model ← a random assignment of *true/false* to the symbols in *clauses*
for each *i* = 1 **to** *max_flips* **do**
 if *model* satisfies *clauses* **then return** *model*
 clause ← a randomly selected clause from *clauses* that is false in *model*
 if RANDOM(0, 1) ≤ *p* **then**
 flip the value in *model* of a randomly selected symbol from *clause*
 else flip whichever symbol in *clause* maximizes the number of satisfied clauses
return *failure*

Figure 7.18: Thuật toán WALKSAT để kiểm tra tính thỏa mãn bằng cách lật ngẫu nhiên các giá trị.

Chuyển pha trong Bài toán SAT



(a)



(b)

Figure 7.19: Đồ thị xác suất mô tả hiện tượng "chuyển pha" (phase transition) của các câu 3-CNF.

Tác nhân Wumpus Lai (Hybrid Agent)

```

function HYBRID-WUMPUS-AGENT(percept) returns an action
  inputs: percept, a list, [stench,breeze,glitter,bump,scream]
  persistent: KB, a knowledge base, initially the atemporal “wumpus physics”
               t, a counter, initially 0, indicating time
               plan, an action sequence, initially empty

  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  TELL the KB the temporal “physics” sentences for time t
  safe  $\leftarrow \{[x,y] : \text{ASK}(\text{KB}, \text{OK}_{x,y}^t) = \text{true}\}$ 
  if ASK(KB, Glittert) = true then
    plan  $\leftarrow$  [Grab] + PLAN-ROUTE(current, {[1,1]}, safe) + [Climb]
  if plan is empty then
    unvisited  $\leftarrow \{[x,y] : \text{ASK}(\text{KB}, L_{x,y}^{t'}) = \text{false} \text{ for all } t' \leq t\}$ 
    plan  $\leftarrow$  PLAN-ROUTE(current, unvisited  $\cap$  safe, safe)
  if plan is empty and ASK(KB, HaveArrowt) = true then
    possible_wumpus  $\leftarrow \{[x,y] : \text{ASK}(\text{KB}, \neg W_{x,y}) = \text{false}\}$ 
    plan  $\leftarrow$  PLAN-SHOT(current, possible_wumpus, safe)
  if plan is empty then // no choice but to take a risk
    not_unsafe  $\leftarrow \{[x,y] : \text{ASK}(\text{KB}, \neg \text{OK}_{x,y}^t) = \text{false}\}$ 
    plan  $\leftarrow$  PLAN-ROUTE(current, unvisited  $\cap$  not_unsafe, safe)
  if plan is empty then
    plan  $\leftarrow$  PLAN-ROUTE(current, {[1,1]}, safe) + [Climb]
  action  $\leftarrow$  POP(plan)
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t  $\leftarrow$  t + 1
  return action

function PLAN-ROUTE(current, goals, allowed) returns an action sequence
  inputs: current, the agent’s current position
           goals, a set of squares; try to plan a route to one of them
           allowed, a set of squares that can form part of the route

  problem  $\leftarrow$  ROUTE-PROBLEM(current, goals, allowed)
  return SEARCH(problem) // Any search algorithm from Chapter 3

```

Figure 7.20: Một chương trình tác tử lai (hybrid) kết hợp tìm kiếm $\mathcal{A}^*$ và suy diễn logic.

Quản lý Trạng thái Niềm tin

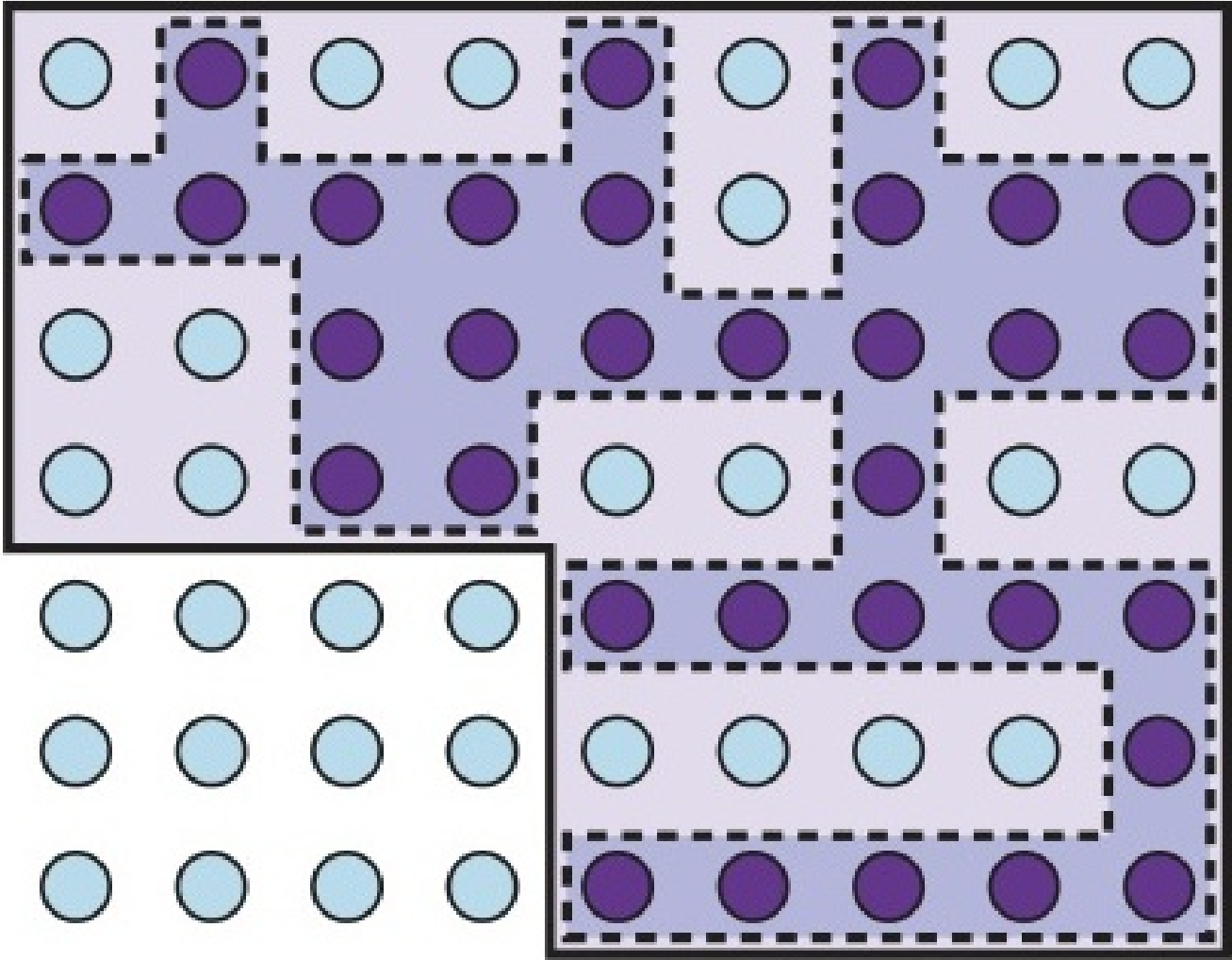


Figure 7.21: Mô tả trạng thái niềm tin 1-CNF như một xấp xỉ bảo thủ thay vì trạng thái chính xác.

Lập kế hoạch với SAT (SATPLAN)

function SATPLAN(*init*, *transition*, *goal*, $T_{\max}$) **returns** solution or *failure*

inputs: *init*, *transition*, *goal*, constitute a description of the problem

$T_{\max}$, an upper limit for plan length

for $t = 0$ **to** $T_{\max}$ **do**

$cnf \leftarrow \text{TRANSLATE-TO-SAT}(init, transition, goal, t)$

$model \leftarrow \text{SAT-SOLVER}(cnf)$

if $model$ is not null **then**

return EXTRACT-SOLUTION($model$)

return *failure*

Figure 7.22: Thuật toán SATPLAN - Dịch bài toán lập kế hoạch thành bài toán SAT để giải quyết.

Tóm tắt Chương 7

- ◇ **Tác nhân Tri thức (KBA):** Có khả năng lưu trữ thông tin về môi trường và suy luận ra các thông tin ẩn nhờ thuật toán Logic.
- ◇ **Wumpus World:** Là môi trường kinh điển đòi hỏi khả năng khám phá và lập luận an toàn dưới dạng quan sát một phần.
- ◇ **Logic Mệnh đề (Propositional Logic):** Biểu diễn sự thật bằng các biến Đúng/Sai kết hợp qua các cổng $\wedge, \vee, \neg, \Rightarrow, \Leftrightarrow$.
- ◇ **Chứng minh Định lý:** Có thể thực hiện bằng cách vét cạn Bảng chân trị (Chậm) hoặc bằng Thuật toán Phân giải CNF - Phản chứng (Nhanh, Hiệu quả cao).
- ◇ **Điểm yếu cực hạn:** Logic mệnh đề cứng nhắc, không thể viết các luật chung chung quy mô lớn (như "Mọi ô có gió thì ô cạnh bên có hố"). Ta cần một ngôn ngữ bậc cao hơn!